Q3. [15 pts] Nearest Neighbors and Bayes risk

In this problem, we want to investigate whether given enough training examples, the Bayes decision rule gives more accurate results that nearest neighbors.

A life insurance company needs to estimate whether a client is at risk of dying in the year to come, based on his age and blood pressure. We call $\mathbf{x} = [A, B]$ (A=Age, B=Blood pressure) the two dimensional input vector and y the outcome ($y = 1$ if the client dies and $y = -1$ otherwise). The insurance company has a lot of data, enough to estimate accurately with Parzen windows the posterior probability $P(y = 1|\mathbf{x})$. This is represented in a diagram in Figure 1.

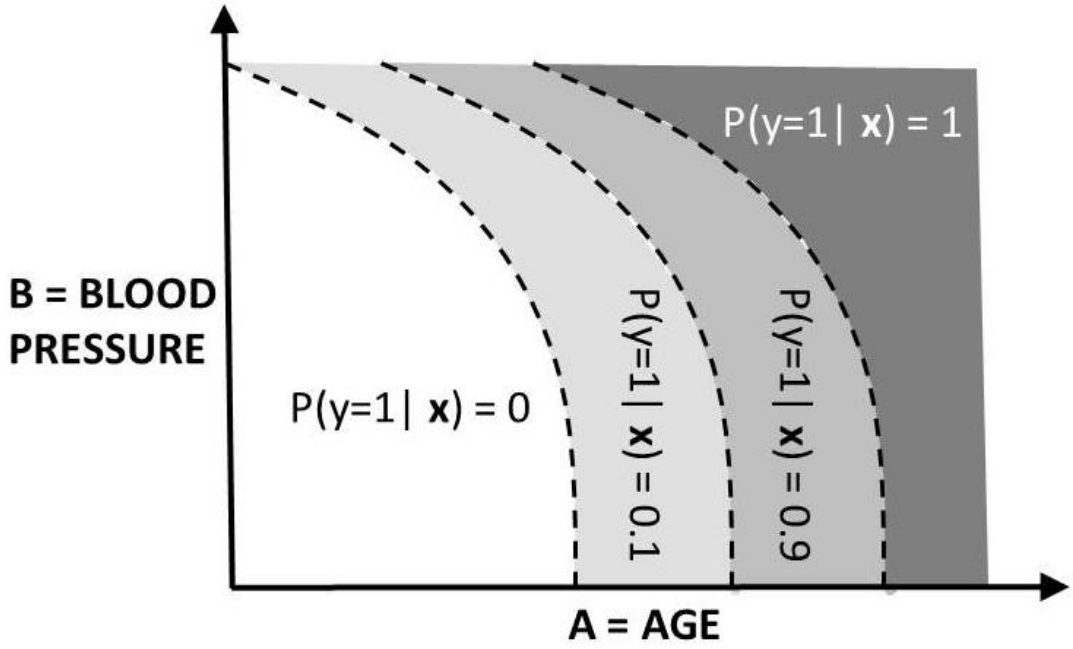


Figure 1: Draw your answer to the Nearest Neighbor and Bayes risk problem. Note that the distribution $P(x)$ is assumed to be uniform across the area shown in the figure.

Note: No worries, nobody died, this is a fictitious example. For simplicity we have 4 regions in which $P(y = 1|\mathbf{x})$ is constant, and the distribution $P(x)$ is assumed to be uniform across the area shown in the figure.

Let us name the different regions:

$$R1 : P(y = 1|\mathbf{x}) = 0$$
$$R2 : P(y = 1|\mathbf{x}) = 0.1$$
$$R3 : P(y = 1|\mathbf{x}) = 0.9$$
$$R4 : P(y = 1|\mathbf{x}) = 1$$

(1) [2 pts] Draw on the figure the Bayes optimum decision boundary with a thick line.

(2) [2 pts] What is the Bayes risk in each of the four regions (the Bayes risk is the probability of error of the optimum Bayes classifier). R1: EBayes = 0
R2: EBayes = 0.1
R3: EBayes = 0.1
R4: EBayes = 0

(3) [4 pts] Assume we have lots and lots of samples due to which we can assume that the nearest neighbor of any sample lies in the same region as that sample. Now consider any sample, say, $\mathbf{x}$ which falls in region R_i. For $i \in \{1, 2, 3, 4\}$, find the probability that $\mathbf{x}$ and its nearest neighbor belong to different classes (that is, have

different labels):

1. If they both fall in R1: 0
2. If they both fall in R2: x 0.9 (+1) x 0.1 (+1) + x 0.1 (+1) x 0.9 (+1) $\Rightarrow$ 2 * 0.1 * 0.9 = 0.18
3. If they both fall in R3: same as R2: 2 * 0.1 * 0.9
4. If they both fall in R4: 0

(4) [2 pts] What is the nearest neighbor error rate ENN in each region:

R1: ENN = 0
R2: ENN = 0.18
R3: ENN = 0.18
R4: ENN = 0

(5) [5 pts] Now let us generalize the previous results to the case where the posterior probabilities are:

$$R1 : P(y = 1|\mathbf{x}) = 0$$
$$R2 : P(y = 1|\mathbf{x}) = p$$
$$R3 : P(y = 1|\mathbf{x}) = (1 - p)$$
$$R4 : P(y = 1|\mathbf{x}) = 1$$

where p is a number between 0 and 0.5.
After recalculating the results of the previous questions, give an upper bound and a lower bound of ENN in terms of EBayes.

$$R1 : EBayes = 0 ENN = 0$$
$$R2 : EBayes = p ENN = 2p(1 - p)$$
$$R3 : EBayes = p ENN = 2p(1 - p)$$
$$R4 : EBayes = 0 ENN = 0$$

EBayes <= ENN <= 2 EBayes(1-EBayes)

Q5. [22 pts] Decision Trees and Random Forests

Consider constructing a decision tree on data with d features and n training points where each feature is real-valued and each label takes one of m possible values. The splits are two-way, and are chosen to maximize the information gain. We only consider splits that form a linear boundary parallel to one of the axes. In parts (a), (b) and (c) we will consider a standalone decision tree and not a random forest, so no randomization.

(1) [4 pts] Prove or give a counter-example: For every value of $m > 3$, there exists some probability distribution on m objects such that its entropy is less than -1. False. The entropy is always non-negative since $-p \log p$ is non-negative when $p \in [0, 1]$.

(2) [4 pts] Prove or give a counter-example: In any path from the root split to a leaf, the same feature will never be split on twice.

False. Example: one dimensional feature space with training poins of two classes x and o arranged as xxxooooxxx.

(3) [4 pts] Prove or give a counter-example: The information gain at the root is at least as much as the information gain at any other node.

False. Example: the XOR function.

(4) [4 pts] One may be concerned that the randomness introduced in random forests may cause trouble, for instance, some features or samples may not be considered at all. We will investigate this phenomenon in the next two parts.

Consider n training points in a feature space of d dimensions. Consider building a random forest with t binary trees, each having exactly h internal nodes. Let f be the number of features randomly selected at each node. In order to simplify our calculations, we will let $f = 1$. For this setting, compute the probability that a certain feature (say, the first feature) is never considered for splitting.

The probability that it is not considered for splitting in a particluar node of a particular tree is $1 - \frac{1}{d}$. The subsampling of $f = 1$ features at each node is independent of all others. There are a total of th nodes and hence the final answer is $(1 - \frac{1}{d})^{th}$.

(5) [3 pts] Now let us investigate the concern regarding the random selection of the samples. Suppose each tree employs n bootstrapped training samples. Compute the probability that a particular sample (say, the first sample) is never considered in any of the trees.

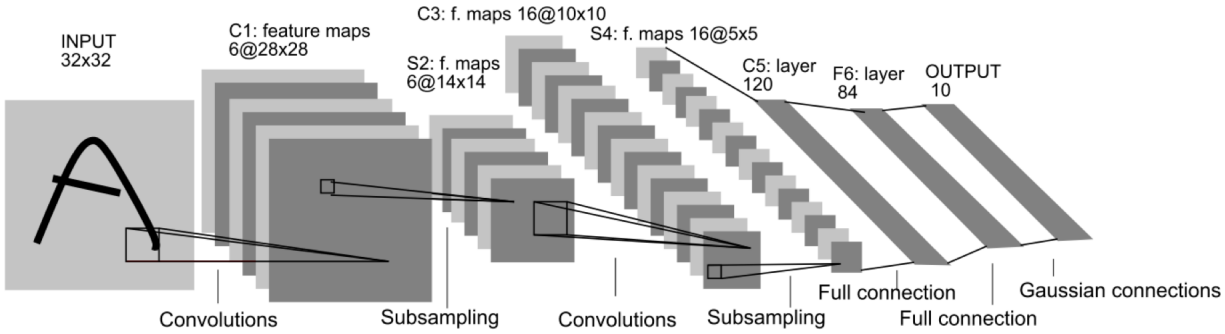
The probability that it is not considered in one of the trees is $(1 - \frac{1}{n})^n$. Since the choice for every tree is independent, the probability that it is not considered in any of the trees is $(1 - \frac{1}{n})^{nt}$.

(6) [3 pts] Compute the values of the probabilities you obtained in the previous two parts for the case when there are $n = 2$ training points, $d = 2$ dimensions, $t = 10$ trees of depth $h = 4$ (you may leave your answer in a fraction and exponentiated form, e.g., as $\left(\frac{51}{100}\right)^2$. What conclusions can you draw from your answer with regard to the concern mentioned in the beginning of the problem?

$\frac{1}{2^{150}}$ and $\frac{1}{2^{20}}$. It is quite unlikely that a feature or a sample will be missed.

Q7. [14 pts] Neural Networks

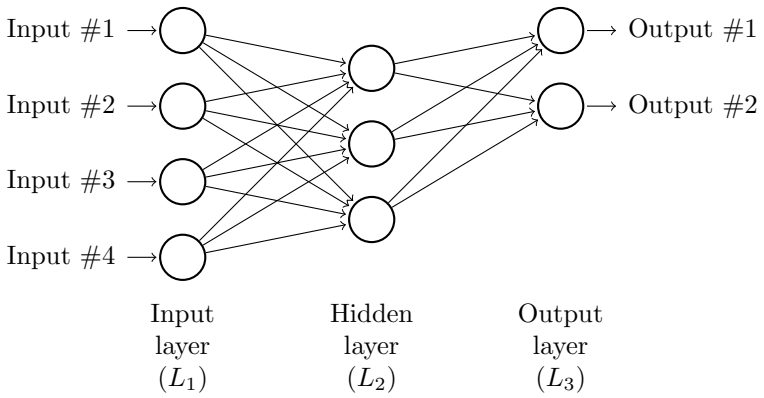
(1) [6 pts] Here is the historical *LeNet* Convolutional Neural Network architecture of Yann LeCun et al. for digit classification that we've discussed in class. Here, the INPUT layer takes in a 32x32 image, and the OUTPUT layer produces 10 outputs. The notation 6@28x28 means 6 matrices of size 28x28.



If the parameters of a given layer are the weights that connect to its inputs,

- Given that the input size is 32x32, and the Layer 1 size is 28x28, what's the size of the convolutional filter in the first layer (i.e. how many inputs is each neuron connected to)? 5×5
- How many independent parameters (weight and bias) are in layer C1? $5 \times 5 \times 6 \times 1 + 6 = 156$
- How many independent parameters (weight and bias) are in layer C3? $5 \times 5 \times 16 \times 6 + 16 = 2416$
- How many independent parameters (weight and bias) are in layer F6? $120 \times 84 + 84 = 10164$

(2) [8 pts] Consider a three layer fully-connected network with n_1, n_2, n_3 neurons in three layers respectively. Inputs are fed into the first layer. The loss is mean squared error E, and the non-linearity is a sigmoid function. Let the label vector be t of size n_3. Let each layer output vector be y_i and input vector be z_i, both of size n_i. Let the weight between layer i and layer $i+1$ be W_{ii+1}. The j-th element in y_i is defined by y_i^j, same for z_i^j. The weight connecting k-th and l-th neuron in $i, i+1$ layers is defined by W_{ii+1}^{kl} (You don't need to consider bias in this problem).



Here is a summary of our notation:

- σ denotes the activation function for L_2 and L_3, $\sigma(x) = \frac{1}{1+e^{-x}}$. There is no activation applied to the input layer.
- $z_i^{(j)} = \sum_{k=1}^{P} W_{i-1i}^{kj} x_{i-1}^{(k)}$
- $y_i^{(j)} = \sigma\left(\sum_{k=1}^{P} W_{i-1i}^{kj} x_{i-1}^{(k)}\right)$

Now solve the following problems.

- Find $\frac{\partial E}{\partial z_3^j}$ in terms of y_3^j.
  $-2y_3^j(1-y_3^j)(t^j - y_3^j)$
- Find $\frac{\partial E}{\partial y_2^k}$ in terms of elements in W_{23} and $\frac{\partial E}{\partial z_3^j}$.
  $\sum_{j=1}^{n_3} W_{23}^{kj} \frac{\partial E}{\partial z_3^j}$
- Find $\frac{\partial E}{\partial W_{23}^{kj}}$ in terms of y_2^k, y_3^j and t^j.
  $y_2^k \frac{\partial E}{\partial z_3^j} = -2y_3^j(1-y_3^j)(t^j - y_3^j)y_2^k$
- If the input to a neuron in max-pooling layer is x and the output is $y = max(x)$, derive $\frac{\partial y}{\partial x_i}$.
  $\frac{\partial y}{\partial x_i} = 1$ if and only if $x_i = max(x)$, otherwise $\frac{\partial y}{\partial x_i} = 0$.